

PHARMACY MANAGEMENT SYSTEM

Expanded Technical & Product Catalog

Features · Architecture · Domain Rules · API · Security · UX · Ops

Version 1.1 Expanded · 2026-08-19

github.com/yasin6606/pharmacy-management-system

Author: yasin

Table of Contents

1. Executive Overview
2. Product Goals & Domain Context
3. Feature Catalog (Deep Dive)
4. System Architecture
5. Technology Stack (Detailed)
6. Backend Architecture & Modules
7. Frontend Architecture & UX Design System
8. Data Model, Entities & Persistence
9. Security, Auth & Access Control
10. Sales, Insurance, Credits & POS Logic
11. Integrations (Titak, Insurance, POS)
12. Error Handling, Logging & Observability
13. Infrastructure, Docker & Deployment
14. API Reference (Expanded)
15. Configuration & Environment Variables
16. Operational Runbook & Incident Response
17. Testing Strategy & Quality
18. Roadmap & Extension Points
19. Glossary & Conventions
- Appendix A. Repository Map
- Appendix B. Example Workflows

1. Executive Overview

The **Pharmacy Management System** is a production-oriented, multi-branch software platform for pharmacy chains and independent pharmacies operating in Iran. It consolidates inventory (batch and expiry), retail sales in **Iranian Rial (IRR)**, patient insurance cost-sharing, card-terminal (POS) settlement, employee management with role-based access, loss reporting, purchasing hooks, analytical reporting, and third-party integrations (Titak price feeds; insurer credential storage for Tamin, Salamat, and Mosalah).

Architecturally the system is a classic three-tier web application: a Next.js browser client, an Express TypeScript API, and PostgreSQL as the system of record, fronted by Nginx and optionally backed by Redis for shared rate limiting. The design favors explicit domain rules, transactional stock safety, and observable failures (structured logs and request correlation IDs).

1.1 Document scope

This expanded catalog documents product features, technical architecture, domain algorithms (sales, insurance, POS), data model, security, API surface, UX design decisions, deployment topology, operations, testing, and extension points. It is intended for engineers, technical leads, and pharmacy IT stakeholders.

1.2 Stakeholders

Stakeholder	Primary needs	Key modules
Owner / manager	Policy, multi-branch control, integrations, franchise	branches, settings, employees, reports
Pharmacist / cashier	Fast accurate sales, stock visibility, insurance, POS	sales, inventory, POS
Accountant	Credits, revenue, exportable reports	credits, reporting, sales summary
Warehouse staff	Transfers, batch intake, expiry awareness	inventory batches, transfer
Engineer / SRE	Deploy, logs, errors, schema, scaling	infra, logger, errorHandler

1.3 Design principles

- Domain correctness over convenience — stock never goes negative under concurrent cashiers
- Money in whole IRR units (rounded integers) for retail display and persistence
- Insurance is opt-in per sale and eligibility-driven per drug line
- Secrets stay server-side; UI only shows masks
- One public network entry (Nginx); data plane remains private
- Every API error is structured and correlatable via requestId

2. Product Goals & Domain Context

2.1 Product goals

- Single system of record for employees, branches, drugs, batches, movements, and sales
- Deterministic stock under concurrent sales and transfers (DB transactions + row locks)
- Native IRR presentation across dashboard, POS, credits, reports, and batch prices
- Insurance-aware checkout: only formulary-eligible drugs share cost with the insurer
- Operator-managed integration credentials (Titak, insurers) without application redeploy
- Bilingual English/Persian experience and light/dark themes including pre-login screens
- Container-first deployment suitable for VPS and Oracle Cloud Always Free ARM instances

2.2 Iranian pharmacy domain

Retail pharmacies in Iran frequently combine cash/card settlement with social insurance cost sharing. Patients present a booklet or electronic member identifier. Not every SKU is reimbursable; the system therefore models **insuranceEligible** and optional **insuranceCode** on the drug master. Coverage percentage is configurable (default 70%) because contracts differ by fund and pharmacy.

Price regulation and distributor catalogs may be synchronized from services such as Titak. The drug record stores **titakCode** as the external identifier. Card payments require a terminal session with an acquirer (e.g. BehPardakht / Beh Mellat style adapters); the POS module models initiate → confirm before finalizing the basket sale.

2.3 Non-goals (current scope)

- Full electronic claim settlement against live insurer networks is adapter-ready, not turnkey without credentials
- Multi-tenant SaaS isolation across unrelated pharmacy companies is not modeled
- Mobile native apps — web responsive UI only
- Automated purchasing replenishment algorithms (hooks exist; optimization is future work)

3. Feature Catalog (Deep Dive)

3.1 Platform bootstrap & identity

A fresh database has no users. The public **Setup** flow creates the first manager (email, password, full name). Subsequent authentication uses JWT. Employee sessions record login metadata for audit. Managers assign roles and current branches; branch history captures assignment changes over time.

Role	Typical permissions (application-level)
junior	Sell, view branch stock; limited administrative writes
senior	Stock adjustments/transfers where gated; broader sales ops
manager	Employees, branches, settings, integrations, full inventory
accountant	Cross-branch sales views, credits, reporting exports

3.2 Inventory & drug catalog

Drugs are master data independent of physical stock. Batches bind a drug to a branch with quantity, expiry, purchase and selling prices, and offer flags. Searching supports name, brand, and company. Deletion of a drug is refused while any related batch still has positive count, preventing orphan stock history.

- Catalog stats: total drugs, total units, active batches — used by dashboard KPIs
- Branch inventory listing filters batches for the cashier's current branch
- Titak update action refreshes selling prices when API key and titakCode are configured
- Expiry awareness: UI highlights near-expiry batches; background job logs alerts

3.3 Sales desk

Cashiers open multiple patient tabs. Each tab holds a basket of batch lines with quantities. Payment method selection drives extra fields: credit captures customer identity; POS requires a successful terminal confirmation; insurance requires provider + member ID and computes coverage on eligible lines only.

3.4 Credits & records

Credit sales remain unpaid until an operator marks the basket paid. The credits UI groups lines by basketId, supports search by customer name/phone, and shows IRR totals. Sales records list historical lines with payment method and branch filters for privileged roles.

3.5 Loss reports & purchasing

Loss reports follow create → approve/reject. Purchasing includes supplier and purchase-order entities plus an OCR client stub for invoice image ingestion — intended as an extension seam rather than a fully automated AP system.

3.6 Reporting

Sales reports accept date range and branch filters, paginate results, and export CSV or PDF. Revenue columns use IRR formatting on the frontend.

3.7 UX product features

- Glassmorphism surfaces, medical teal primary, gold accent, responsive sidebar
- next-intl locale prefix always-on (/en/..., /fa/...)
- Theme and language controls on login and setup
- Severity-colored toasts; optional requestId for support desks

4. System Architecture

4.1 Context diagram (textual)

Human users operate a browser. The only host-published service is Nginx. Nginx routes API and health traffic to the backend container and page traffic to the frontend container. Backend talks to PostgreSQL for persistence and Redis for distributed rate-limit counters when scaling API replicas. Outbound HTTPS may reach Titak or future insurer endpoints using stored credentials.

Inbound path

```

Client Browser
  ■ HTTP :80 (or HTTP_PORT)
  ▼
Nginx (edge)
  ■■ /api/* , /health → backend:3001
  ■■ /*              → frontend:3000
backend → postgres:5432
backend → redis:6379
backend → external APIs (Titak, insurers, POS acquirer)

```

4.2 Layering

Layer	Location	Responsibility
Presentation	frontend/	Routes, forms, design system, i18n, client state
API / Application	backend/src/modules	Use-cases, validation, authz, DTOs
Domain services	*.service.ts	Transactions, rules, aggregation
Persistence	TypeORM entities/repos	SQL mapping, locks
Infrastructure	core/, Docker, Nginx	Logging, errors, networking

4.3 Trust boundaries

- Browser is untrusted — all authorization enforced on the API
- Nginx terminates public traffic; backend/frontend not published by default
- Postgres/Redis bind only to internal Compose network
- Integration secrets never returned in full to the browser

4.4 Scalability notes

Stateless API containers can scale horizontally if JWT validation remains local and Redis backs rate limits. POS in-memory sessions are single-node today — multi-replica POS requires a shared store (documented roadmap). PostgreSQL remains the write authority for stock.

5. Technology Stack (Detailed)

Area	Choice	Rationale
UI framework	Next.js App Router + React	SSR/SSG capable SPA routes, modern React
Language	TypeScript (strict-ish)	Safer refactors across FE/BE
Styling	Tailwind + CSS variables	Fast design tokens; light/dark
i18n	next-intl	Locale routing + message catalogs
Theme	next-themes	Class-based dark mode
Forms	react-hook-form + Zod	Client validation aligned with server Zod
HTTP client	Axios	Interceptors for auth and errors
API server	Express	Mature middleware ecosystem
ORM	TypeORM	Decorators, transactions, pessimistic locks
DI	Awilix	Composable service wiring
Logging	Winston	Levels, JSON, optional files
Auth crypto	bcryptjs + jsonwebtoken	Password hash + bearer tokens
DB	PostgreSQL 16	Reliability, SQL, concurrent writes
Cache	Redis 7	Shared rate limits
Edge	Nginx	Path routing, future TLS
Containers	Docker multi-stage	Small runtime images, reproducible builds

6. Backend Architecture & Modules

6.1 Process bootstrap

index.ts initializes TypeORM DataSource, creates the Express app via createApp(), listens on PORT (default 3001), starts the expiration alert job, and registers process handlers for unhandledRejection and uncaughtException so failures are logged before exit.

6.2 Middleware pipeline

- helmet — security headers
- cors — cross-origin (restrict CORS_ORIGIN in production)
- express.json limit 1mb — body parsing
- cookieParser — cookie support
- requestLogger — assigns x-request-id, logs duration/status
- Route modules under /api/v1
- API 404 JSON for unknown /api paths
- errorHandler — last; maps errors to structured JSON

6.3 Module catalog

Module	Entrypoints	Responsibilities
setup	POST /setup	Create first manager if empty
auth	/auth/*	Login, me, password, sessions
employees	/employees	CRUD, roles, branch assignment
branches	/branches	CRUD, franchise flag
inventory	/inventory/*	Drugs, batches, transfer, stats
sales	/sales/*	Batch sale, list, summary, pay credit
settings	/settings/*	Franchise amount, integrations KV
titak	/integrations/titak/*	Fetch/apply external prices
pos	/integrations/pos/*	Terminal session lifecycle
insurance	/integrations/insurance/*	Adapter validate/submit hooks
loss-reports	/loss-reports	Submit & review workflow
reporting	/reporting/*	Aggregations + CSV/PDF
purchasing	/purchasing/*	Suppliers, POs, OCR client

6.4 Transactional patterns

Inventory-affecting use cases wrap work in AppDataSource.transaction. Critical reads use pessimistic_write locks on DrugBatch to serialize concurrent cashiers selling the same lot. Failures throw AppError with 4xx codes so the client receives actionable messages without partial stock updates.

6.5 Validation

Write endpoints use Zod schemas (DTO modules). Failures become VALIDATION_ERROR with per-field paths in the error handler. This keeps transport validation consistent and reduces ad-hoc checks.

7. Frontend Architecture & UX Design System

7.1 Routing structure

All user-facing routes live under `app/[locale]/...` with locales `en` and `fa`. Auth group hosts login and setup. Dashboard group hosts operational screens and shares a layout with sidebar, error toast host, and auth gate.

7.2 Design tokens & glass UI

`globals.css` defines semantic colors via CSS variables for light and dark schemes. Utility classes `.glass` and `.glass-strong` apply translucent backgrounds, blur, and borders. Primary brand color is medical teal; accent gold highlights secondary actions. Motion respects reduced-motion preferences where implemented.

7.3 Component library

Component	Role
Button	Variants default/outline/ghost; sizes including icon
Input / Select	Labeled fields, error text
Card	Section containers
Table	Data grids with horizontal scroll on small screens
Modal	Dialogs for create/edit flows
Sidebar	Role-filtered navigation + theme/language
Pagination	Page controls for list APIs
ErrorToast	Severity-colored alerts with optional requestId
ThemeToggle / LanguageSwitcher	Global preferences

7.4 State & data fetching

`AuthContext` holds token/user; `sessionStorage` persists JWT. `SalesTabsContext` keeps multi-basket state for the sales desk. `useApi` wraps `apiGet/Post/...` and routes failures into `ErrorContext`. `formatIRR` in `lib/currency.ts` centralizes money display for `fa/en` locales.

7.5 Screen inventory

Screen	Path (pattern)	Notes
Login	<code>/[locale]/login</code>	Theme + language controls
Setup	<code>/[locale]/setup</code>	First manager bootstrap
Dashboard	<code>/[locale]/dashboard</code>	KPI cards from summary APIs
Drugs	<code>/[locale]/inventory/...</code>	Catalog CRUD, Titak, insurance flags
Batches	<code>/[locale]/inventory/batches</code>	Stock, transfer, price refresh
Sales desk	<code>/[locale]/sales</code>	Multi-tab basket POS
Sales records	<code>/[locale]/sales/records</code>	History + filters
Credits	<code>/[locale]/credits</code>	Unpaid baskets, mark paid
Reports	<code>/[locale]/reports</code>	Filters + CSV/PDF
Employees	<code>/[locale]/employees</code>	Staff management
Branches	<code>/[locale]/branches</code>	Sites + franchise
Loss reports	<code>/[locale]/loss-reports</code>	Workflow
Settings	<code>/[locale]/settings</code>	Password, franchise, integrations

8. Data Model, Entities & Persistence

8.1 Entity dictionary

Entity	Key fields	Notes
Employee	email, passwordHash, role, currentBranchId, fullName	Auth principal
EmployeeSession	employeeId, loginAt, logoutAt, ip	Audit sessions
EmployeeBranchHistory	employeeId, branchId, assignedAt	Assignment trail
Branch	name, address, isWarehouse, hasFranchise	Org unit
Drug	name, brand, company, titakCode, insuranceEligible, insuranceCode, lastPriceUpdateDate	Catalog
DrugBatch	drugId, branchId, expirationDate, count, purchasePrice, sellingPrice, isOffer	Physical stock
StockMovement	type, quantity, from/to branch, performedBy, note	Ledger
SaleTransaction	qty, unitPrice, totalPrice, paymentMethod, franchiseFee, insurance*, patientShare, basketId, isPaid, customer*	Sale line
Settings	key, value (numeric)	e.g. franchise_amount
IntegrationSetting	key, value (string)	Secrets & URLs
LossReport	status, quantities, reviewer fields	Workflow
Supplier	name, contact	Purchasing
PurchaseOrder	supplier, status, totals	Purchasing

8.2 Relationships (conceptual)

Branch 1—* DrugBatch; Drug 1—* DrugBatch; DrugBatch 1—* SaleTransaction; Employee 1—* SaleTransaction; SaleTransaction lines sharing basketId form a patient basket. StockMovement references batches and optional from/to branches for transfers.

8.3 Money & quantities

Quantities are integers. Prices and totals are stored as numbers intended as whole IRR (application rounds with Math.round on sale). Frontend formatIRR applies locale grouping and Rial labels.

8.4 Schema evolution

Docker defaults TYPEORM_SYNCHRONIZE=false. Development may enable synchronize for speed; production should apply migrations under backend/src/migrations and keep synchronize off.

9. Security, Auth & Access Control

9.1 Authentication flow

POST /auth/login validates credentials, compares bcrypt hash, issues JWT embedding userId, role, and branchId claims, and records a session. Clients store the token in sessionStorage and send Authorization: Bearer on subsequent calls. GET /auth/me rehydrates the user for the SPA shell.

9.2 Authorization

rbac middleware checks required roles per route. Service-level filters further restrict data: non manager/accountant sales queries force employeeId/branchId from the token. Settings integrations are manager-gated.

9.3 Threat mitigations

Threat	Mitigation
Password theft at rest	bcrypt hashing
Credential stuffing	Rate limit (Redis/memory) on login
XSS token theft	Avoid localStorage long-term; sessionStorage + short-lived JWT policy
CSRF	Bearer header API (not cookie session for API auth)
Injection	TypeORM parameters + Zod validation
Secret leakage in UI	Masked integration values; empty field keeps previous secret
Lateral movement to DB	DB not published; internal Docker network only

9.4 Security headers & transport

Helmet sets standard browser security headers. Production deployments should terminate TLS at Nginx or a cloud load balancer and set CORS_ORIGIN to the real web origin.

10. Sales, Insurance, Credits & POS Logic

10.1 Batch sale algorithm (detailed)

Input: items[{drugBatchId, quantity, prescriptionRef?}], payment{method, customer*, posReference?, insuranceProvider?, insuranceMemberId?}.

- Reject empty items or missing branch on the authenticated user
- Normalize insuranceProvider; if not none, require non-empty insuranceMemberId
- Load coverage percent from settings when insurance active (else 0)
- Begin DB transaction
- Load franchise setting and branch.hasFranchise
- For each item: lock batch row; verify existence, stock, branch ownership; decrement count
- Compute lineTotal = round(sellingPrice) * qty
- If drug.insuranceEligible and provider set: coverage = round(lineTotal * percent/100); else 0
- patientShare = lineTotal - coverage; insert SaleTransaction + StockMovement SALE
- Apply franchise fee once on first sale line if configured; add to patient share totals
- Return {basketId, currency:IRR, insurance totals, coveragePercent}

10.2 Insurance business rules

Important: Cash-only drugs (insuranceEligible=false) never reduce patient liability even when a provider is selected. This mirrors formulary behavior and prevents accidental insurer billing on non-covered SKUs.

10.3 POS lifecycle

POS is mandatory confirmation when paymentMethod=pos. Frontend disables Complete Sale until status is approved. Backend PosService.initiate(amount) creates a reference via BehMellatAdapter (stub success with BM-timestamp codes) and stores a pending session with TTL. confirm(reference, approved) transitions state; approved reference is stored on the sale as posReference.

Cashier selects POS

```
→ POST /integrations/pos/initiate { amount: patientShare }
← { referenceCode, amount, currency: IRR, status: pending }
→ Customer pays on terminal
→ POST /integrations/pos/confirm { referenceCode, approved: true }
→ POST /sales/batch { ... payment: { method: 'pos', posReference } }
```

10.4 Credits

method=credit sets isPaid=false. PATCH /sales/basket/:basketId/pay marks all lines in the basket for the user's branch as paid. UI groups by basket and shows unpaid IRR totals.

10.5 Summaries

GET /sales/summary aggregates SUM(totalPrice) and COUNT with the same filters as list endpoints, avoiding the classic bug of summing only the current page.

11. Integrations (Titak, Insurance, POS)

11.1 Settings-backed configuration

IntegrationSetting is a string key/value table. Managers PUT /settings/integrations with a map of keys. Secret-like keys are masked on GET. Submitting an empty string for a secret key leaves the stored value unchanged.

Key	Purpose	Secret?
titak_api_key	Titak HTTP API key	yes
titak_base_url	Override default Titak base URL	no
insurance_tamin_api_key	Tamin credentials	yes
insurance_salamat_api_key	Salamat credentials	yes
insurance_mosalah_api_key	Mosalah credentials	yes
insurance_default_coverage_percent	Default share e.g. 70	no
pos_terminal_id	Default terminal id for POS	no

11.2 Titak service behavior

Resolves API key from settings then env TITAK_API_KEY. Uses drug.titakCode when present. On success, updates drug.lastPriceUpdateDate and related batch selling prices. Failures surface as operational errors to the client.

11.3 Insurance adapters

BaseInsuranceAdapter defines validateMember / submitClaim style methods. Sample adapters demonstrate the seam. Production pharmacies plug in real HTTP clients using the stored keys and their vendor documentation — the core sale path does not block on live claim APIs today; it records coverage math locally for POS/cash reconciliation.

11.4 POS adapters

PosAdapter abstract class: initiatePayment, checkStatus. BehMellatAdapter currently returns synthetic success references suitable for integration testing until the real SDK/IPC bridge is configured on pharmacy hardware.

12. Error Handling, Logging & Observability

12.1 Backend error contract

```
{ "success": false, "code": "VALIDATION_ERROR", "message": "...",  
  "details": [{ "path": "email", "message": "..."}], "requestId": "uuid" }
```

Source	HTTP	code	Notes
ZodError	400	VALIDATION_ERROR	Field-level details array
AppError	varies	err.code	Operational messages
QueryFailedError	400	DATABASE_ERROR	Generic msg in prod
JWT errors	401	UNAUTHORIZED	Invalid/expired token
Bad JSON body	400	BAD_REQUEST	SyntaxError with body
Unknown	500	INTERNAL_ERROR	Stack logged server-side

12.2 AppError helpers

Factories: badRequest, unauthorized, forbidden, notFound, conflict, tooManyRequests, internal. Constructor remains backward compatible with (message, status, isOperationalBoolean).

12.3 Logging

Winston logger: defaultMeta service=pharmacy-api; level from LOG_LEVEL; pretty console in development; JSON in production; optional rotating files when LOG_TO_FILES=true. requestLogger emits method, path, status, durationMs, userId, requestId. Health checks log at debug.

12.4 Frontend observability

ApiError carries status, code, requestId. clientLog writes debug/info only in development. useApi maps status codes to user-facing strings, suppresses 401 toasts, deduplicates messages, and caps visible toasts. ErrorToast shows code + optional ref for support.

12.5 Correlation playbook

1) Capture requestId from toast or response header x-request-id. 2) docker compose logs backend | grep requestId. 3) Inspect parallel DB state if stock-related.

13. Infrastructure, Docker & Deployment

13.1 Compose topology

Service	Image	Published?	Health
postgres	postgres:16-alpine	no	pg_isready
redis	redis:7-alpine	no	redis-cli ping
backend	build ../backend	no	HTTP /health (service config)
frontend	build ../frontend	no	service healthcheck
nginx	nginx:1.27-alpine	yes :HTTP_PORT	depends on FE/BE healthy

13.2 Build characteristics

Backend Dockerfile multi-stage: install, webpack build, runtime node image, non-root where configured. Frontend builds Next.js with NEXT_PUBLIC_API_URL=/api/v1 so the browser calls same-origin paths through Nginx.

13.3 Deployment targets

- Local developer machine via Docker Desktop / Engine
- Generic Linux VPS with Docker Compose
- Oracle Cloud Always Free Ampere A1 — see infrastructure/deploy/oracle-cloud.md and bootstrap scripts

13.4 Day-2 operations

```
docker compose ps
docker compose logs -f backend
docker compose up -d --build backend frontend
docker compose down          # keep volumes
docker compose down -v       # wipe data
```

14. API Reference (Expanded)

Base URL path prefix: /api/v1. Success responses typically { success: true, data }. Auth: Bearer JWT unless setup/login.

14.1 Auth & setup

Method	Path	Description
POST	/setup	Create first manager (empty DB)
POST	/auth/login	Returns token + user
GET	/auth/me	Current user profile
PUT	/auth/change-password	Authenticated password change

14.2 Master data

Method	Path	Description
GET/POST	/employees	List/create staff
GET/PUT	/employees/:id	Read/update staff
GET/POST	/branches	List/create branches
PATCH	/branches/:id/franchise	Toggle franchise flag
GET/POST	/inventory/drugs	List/create drugs
GET/PUT/DELETE	/inventory/drugs/:id	Drug detail lifecycle
GET/POST	/inventory/batches	Batch operations
POST	/inventory/transfer	Inter-branch transfer
GET	/inventory/catalog/stats	Dashboard counters
GET	/inventory/branches/:id/inventory	Branch stock view

14.3 Sales & payments

Method	Path	Description
GET	/sales	Paginated sales lines + filters
GET	/sales/summary	SUM/COUNT aggregates
POST	/sales/batch	Atomic multi-line sale
PATCH	/sales/basket/:id/pay	Mark credit basket paid
POST	/integrations/pos/initiate	Start terminal payment
POST	/integrations/pos/confirm	Approve/decline terminal
GET	/integrations/pos/status/:ref	Session status

14.4 Settings, reports, misc

Method	Path	Description
GET/PUT	/settings/franchise	Franchise amount
GET/PUT	/settings/integrations	Integration key map
POST	/integrations/titak/update-price/:drugId	Pull Titak price
GET	/reporting/sales	Report rows
GET	/reporting/sales/export	CSV/PDF export
GET/POST	/loss-reports	Loss workflow
GET	/health	Liveness (unversioned root)

15. Configuration & Environment Variables

Variable	Component	Required	Description
POSTGRES_USER/PASSWORD/DB	Compose	yes (pwd)	Database bootstrap
JWT_SECRET	Backend	yes prod	Sign/verify tokens
JWT_EXPIRES_IN	Backend	no	Default 7d
DATABASE_URL	Backend	yes	Postgres connection string
REDIS_URL	Backend	no	Rate limit shared store
TYPEORM_SYNCHRONIZE	Backend	no	true only bootstrap/dev
TITAK_API_KEY	Backend	no	Fallback to Settings UI
LOG_LEVEL	Backend	no	debug info warn error
LOG_TO_FILES	Backend	no	Enable file transports
CORS_ORIGIN	Backend	no	Browser origin allowlist
PORT	Backend	no	Default 3001
NEXT_PUBLIC_API_URL	Frontend	yes build	/api/v1 in Compose
HTTP_PORT	Compose	no	Host port for Nginx

16. Operational Runbook & Incident Response

16.1 First production boot

- Clone repo; create infrastructure/.env from example with strong secrets
- Decide schema strategy: temporary TYPEORM_SYNCHRONIZE=true or run migrations
- docker compose up --build -d; wait for healthchecks
- curl /health → {status:ok}
- Open /fa/setup or /en/setup; create manager; log in
- Create at least one retail branch; assign manager branch
- Enter drugs (set Titak/insurance flags); add batches with IRR prices
- Configure integration keys under Settings if available
- Perform a test cash sale and a test POS path in a non-production terminal mode

16.2 Common incidents

Symptom	Likely cause	Actions
npm ci failures in image	Lockfile drift	Use install fallback; refresh lockfile
Today sales = 0	Summary/filter/date bounds	Verify /sales/summary and timezone day bounds
Stock errors on sell	Concurrent sell / wrong branch	Check AppError message; review movements
401 loops	Expired JWT	Re-login; check JWT_SECRET stability
POS cannot complete	Missing confirm	Ensure approved status before batch sale
Titak update fails	Missing key/code	Settings key + drug.titakCode

16.3 Backup & restore

Back up Docker volume postgres_data and/or scheduled pg_dump. Restore into a stopped stack, then bring services up. Redis can be rebuilt empty (rate limits reset).

17. Testing Strategy & Quality

17.1 Backend

Jest suites cover pure utilities (pagination, JWT helpers), middleware (auth, rbac, validation, rateLimit), and services (auth, employees, branches, inventory transfer/expiring, sales, settings, loss-reports, setup). Prefer service tests when changing insurance math or stock locks.

17.2 Frontend

Jest + Testing Library cover ApiError, navigation helpers, useRole, ErrorContext, and utils. Critical user journeys (POS + insurance) are candidates for future Playwright E2E.

17.3 Quality gates

- TypeScript compile on backend webpack build and Next build
- Docker image build as integration smoke test
- Manual checklist: setup → branch → drug → batch → sale → report

18. Roadmap & Extension Points

- Formal TypeORM migrations for every schema change; CI migration check
- Redis (or DB) backed POS session store for multi-replica API
- Production BehMellat/acquirer SDK with terminal status polling
- Live insurer validateMember/submitClaim using stored API keys
- TLS at edge; HSTS; strict CORS_ORIGIN
- Centralized APM/Sentry wiring into Winston and clientLog
- Playwright E2E for basket, insurance, POS, credit mark-paid
- Advanced inventory: min stock alerts, purchase suggestions
- Fine-grained permission matrix beyond four roles if chains require it

19. Glossary & Conventions

Term	Definition
IRR	Iranian Rial; system retail currency
Batch / DrugBatch	Stock lot at a branch with shared expiry and prices
Basket	Set of sale lines sharing basketId for one checkout
Franchise fee	Optional fixed fee when branch.hasFranchise is true
Titak	External drug pricing service integration
Tamin / Salamat / Mosalah	Iranian insurance / social health organizations
RBAC	Role-based access control
requestId	Correlation identifier across client and server logs
AppError	Operational error safe to return to API clients
POS	Point-of-sale card terminal payment flow
Pessimistic lock	DB row lock preventing concurrent conflicting updates
Glass UI	Translucent, blurred panel aesthetic used in the design system

Conventions

- API JSON: camelCase field names
- Dates: ISO-8601 in API; localized in UI
- Money: integer IRR in logic; formatted strings only in UI
- Locales: en, fa — always present as first path segment

Appendix A. Repository Map

```
pharmacy-management-system/
■■■ backend/src/core/           # errors, logger, middleware, config
■■■ backend/src/modules/        # domain modules
■■■ backend/tests/              # Jest tests
■■■ frontend/app/[locale]/      # routes
■■■ frontend/components/        # ui + forms
■■■ frontend/context|hooks|lib  # state & clients
■■■ frontend/messages/         # en.json, fa.json
■■■ docs/                       # catalogs & guides
■■■ infrastructure/             # compose, nginx, deploy scripts
```

Appendix B. Example Workflows

B.1 First-time pharmacy go-live

- Deploy stack → setup manager → create Branch A (retail)
- Create drug Amoxicillin 500mg; mark insuranceEligible; set titakCode if known
- Add batch: qty 100, expiry +1y, sellingPrice in IRR
- Open sales desk → add line → insurance Tamin + member ID → cash complete
- Verify dashboard today's sales and inventory decrement

B.2 POS with partial insurance

- Basket: eligible drug + non-eligible drug
- Provider Salamat → coverage applies only to eligible line
- Patient share = non-covered + patient portion of covered + franchise?
- Initiate POS for patient share → confirm approved → complete sale

B.3 Credit collection

- Sell on credit with customer name/phone
- Credits page → expand basket → Mark as paid
- Verify isPaid and badge state

End of expanded catalog v1.1 — Pharmacy Management System. Source: github.com/yasin6606/pharmacy-management-system · 2026-08-19.